

OPAQUE PIXEL

Advanced Steganography Platform

Software Documentation for Project Submission

Document date: June 26, 2026

Domain: opaquepixel.app

This document describes the design, architecture, features, and operation of the Opaque Pixel web platform. It is intended for academic submission and as source material for PowerPoint presentation slides.

1. Abstract

Opaque Pixel is a web-based steganography platform that allows authorized users to hide encrypted secret data inside ordinary digital files such as images, videos, and documents. The system combines modern cryptography (AES-256-GCM), multiple steganographic embedding techniques (LSB, DST, video frame embedding, document carrier injection), and a secure QR-based authentication gate. The platform provides a minimalist dark web interface for non-technical users while maintaining a modular FastAPI backend suitable for research, cybersecurity education, and lawful privacy applications.

2. Introduction

Steganography is the practice of concealing information within other non-secret data. Unlike encryption alone, which makes data unreadable but visible, steganography hides the very existence of the secret message. Opaque Pixel was developed as a complete full-stack application demonstrating practical steganography with strong encryption and user-friendly workflows.

Problem Statement

Users need a reliable way to embed confidential data in everyday file formats without obvious indicators of hidden content, while ensuring only authorized individuals can access the platform and only the correct password can recover the payload.

Objectives

- Build a secure web platform for hiding and revealing encrypted payloads.
- Support multiple carrier types: image, video, and document formats.
- Implement strong encryption before embedding (AES-256-GCM with PBKDF2).
- Provide QR-based authentication for platform access control.
- Deliver an intuitive, modern web interface for educational and research use.

3. System Overview

Opaque Pixel is a client-server application. The React frontend handles user interaction, file uploads, and session management. The FastAPI backend performs encryption, steganographic embedding/extraction, and authentication. All hide and reveal operations require a valid session token obtained through QR verification.

Main Modules

Module	Role
pixelvault-frontend	React UI, routing, auth gate, file upload
pixelvault-backend	REST API, crypto, stego engines
Auth system	Signed QR verification, HMAC session tokens
Crypto engine	AES-256-GCM encryption/decryption
Stego engines	Image, video, document embedding

4. System Architecture

The architecture follows a three-tier model: Presentation (React + Tailwind CSS), Application (FastAPI routers and business logic), and Processing (steganography and cryptography engines). Temporary files are stored during processing and cleaned up after each request.

Data Flow - Hide Operation

- User uploads carrier file and payload (text or file).
- User enters encryption password (minimum 8 characters).
- Backend packages payload with metadata and compresses if needed.
- Payload is encrypted with AES-256-GCM (PBKDF2 key derivation, 600,000 iterations).
- Encrypted blob is embedded into the carrier using the selected stego method.
- Modified carrier file is returned to the user for download.

Data Flow - Reveal Operation

- User uploads stego carrier file and enters password.
- Backend extracts embedded data using appropriate stego engine.
- Data is decrypted; wrong password returns an error.
- Original payload is returned (text, file, or document).

5. Technology Stack

Layer	Technologies
Frontend	React 18, Vite, Tailwind CSS, React Router, Axios
Backend	Python 3, FastAPI, Uvicorn
Image processing	OpenCV, NumPy, SciPy, Pillow
Security	cryptography (AES-GCM), HMAC, PBKDF2, QR codes
Testing	pytest, FastAPI TestClient
Deployment	Nginx, Cloudflare Pages, Railway/Render (optional)

6. Key Features

- Hide encrypted payloads in images (PNG/JPG), videos (MP4), and documents (PDF, DOCX, etc.).
- Reveal hidden payloads with password-protected decryption.
- Dual image algorithms: LSB (fast, lossless PNG) and DST (compression-resistant).
- Document steganography via ZIP injection (Office/ODF) and append method (PDF, TXT, etc.).
- Video steganography using keyframe embedding.
- QR-based authentication with HMAC-signed tokens and rate limiting.
- Capacity estimation bar for image carriers.
- WhatsApp compression warning for image/video carriers.
- Minimalist dark portfolio-style responsive UI.
- Session-based access (24-hour default token lifetime).

7. Steganography Methods

7.1 Image - LSB (Least Significant Bit)

LSB replaces the least significant bits of pixel color channels with encrypted payload bits. Works best on lossless PNG images. Fast and simple but destroyed by JPEG recompression or messaging app compression (e.g. WhatsApp).

7.2 Image - DST (Modified DCT)

DST embeds data in the frequency domain using discrete cosine transform coefficients, making it more resistant to mild compression than LSB. Recommended when the carrier may undergo format conversion or light compression.

7.3 Video Steganography

Video carriers (MP4) embed encrypted data across selected video frames using image steganography techniques applied per frame, with configurable keyframe intervals.

7.4 Document Steganography

Office Open XML formats (DOCX, PPTX, XLSX, ODT, ODS, ODP) are ZIP archives. Payload is hidden inside a dedicated ZIP entry. PDF, RTF, TXT, CSV, MD, HTML, and XML use an append-based method with a magic header marker for extraction.

8. Encryption and Security

All payloads are encrypted before embedding. The system never stores user passwords. Wrong passwords fail decryption without revealing partial data.

Component	Implementation
Algorithm	AES-256-GCM (authenticated encryption)
Key derivation	PBKDF2-HMAC-SHA256, 600,000 iterations
Salt	16 random bytes per encryption
Nonce	12 random bytes (GCM standard)
Session token	HMAC-SHA256 signed JSON payload with expiry
Auth QR	OPX1 signed format; verified with AUTH_SECRET
Rate limiting	8 attempts per 15 minutes on auth endpoint

9. Authentication Flow

- User opens the landing page and reads disclaimer/policy.
- Administrator distributes auth QR code offline (not downloadable from UI).
- User uploads QR image; backend decodes and verifies HMAC signature.
- On success, a bearer session token is issued (default 24-hour TTL).
- Hide and Reveal API endpoints require Authorization: Bearer header.

- Expired or invalid tokens are rejected; user must re-authenticate.

10. User Interface

The frontend uses a minimalist dark portfolio design with glassmorphism cards, purple/blue ambient gradients, animated logo, and cursor-following glow effects. The landing page includes hero, about, pipeline, formats, WhatsApp disclaimer, and access sections. Workspace pages (Hide/Reveal) use a step-by-step card layout.

Pages

Page	Purpose
Info / Landing	Disclaimer, auth QR upload, platform overview
Hide	3-step workflow: Carrier, Payload, Encrypt
Reveal	Upload stego file, algorithm select, decrypt

11. Supported File Formats

Carriers

Images: PNG, JPG, JPEG | Video: MP4 | Documents: PDF, DOCX, PPTX, XLSX, ODT, ODS, ODP, ODG, RTF, TXT, CSV, MD, HTML, XML

Payloads

Text, documents, images (PNG/JPG), video (MP4), audio (MP3)

12. REST API Endpoints

Method	Path	Auth	Description
POST	/api/auth/verify	No	Verify auth QR, return token
POST	/api/hide	Yes	Embed encrypted payload in carrier
POST	/api/reveal	Yes	Extract and decrypt hidden payload
GET	/api/health	No	Health check

13. Installation and Execution

Local Development

Backend: cd pixelvault-backend, activate venv, pip install -r requirements.txt, uvicorn main:app --reload --port 8000
Frontend: cd pixelvault-frontend, npm install, npm run dev
Open http://localhost:5173

Production (Summary)

Build frontend with VITE_API_URL=/api. Deploy static files to Cloudflare Pages. Deploy backend to Railway/Render with

environment secrets. Point DNS opaquepixel.app to frontend and api.opaquepixel.app to backend. Enable HTTPS via platform SSL.

14. Testing

Automated tests cover cryptography roundtrip, LSB/DST image stego, document stego, auth token validation, signed QR verification, and protected endpoint access. Run: pytest in pixelvault-backend directory.

15. Limitations and Disclaimers

- WhatsApp and similar apps recompress photos/videos, destroying hidden data.
- LSB is not suitable for JPEG or compressed images.
- Large video files require significant processing time and memory.
- Platform is for educational, research, and lawful privacy use only.
- Users are responsible for compliance with local laws and policies.
- Auth QR must be kept private; compromise allows platform access.

16. Future Enhancements

- Audio steganography support expansion.
- Batch processing and API key management for researchers.
- Steganalysis detection dashboard for educational comparison.
- Mobile-responsive PWA with offline QR auth cache.
- Audit logging and admin panel for enterprise deployments.

17. Conclusion

Opaque Pixel demonstrates a complete, modern steganography platform combining encryption, multiple embedding techniques, secure authentication, and an accessible web interface. It serves as a practical tool for cybersecurity education, privacy research, and authorized testing while emphasizing lawful and responsible use.

18. Suggested PowerPoint Slide Outline

- Slide 1: Title Slide - Opaque Pixel: Advanced Steganography Platform
- Slide 2: Introduction and Problem Statement
- Slide 3: Objectives and Scope
- Slide 4: System Architecture Diagram
- Slide 5: Technology Stack
- Slide 6: Encryption: AES-256-GCM + PBKDF2
- Slide 7: Steganography Methods (LSB, DST, Video, Document)
- Slide 8: Authentication Flow (QR + Session Token)
- Slide 9: Hide Workflow Demo Screenshots
- Slide 10: Reveal Workflow Demo Screenshots
- Slide 11: Supported Formats Table
- Slide 12: UI Design Overview
- Slide 13: Security Features
- Slide 14: WhatsApp Compression Warning
- Slide 15: Testing and Validation
- Slide 16: Limitations and Ethical Use
- Slide 17: Future Work
- Slide 18: Conclusion and Q&A